

Designing a System That Monetizes Data

Big Data Systems Final Project

b11202010 Han Yu-Min

GitHub Repository:

<https://github.com/MaxHan1120/b11202010-taiwan-tech-risk-dashboard>

Live Demo:

<https://maxhan1120.github.io/b11202010-taiwan-tech-risk-dashboard/>

Contents

1	Introduction	3
2	Target Customer	3
3	Evidence of Demand and Willingness to Pay	4
3.1	Competitor and Analogous Product Benchmarks	4
3.2	Behavioral Evidence	5
3.3	Estimated Time Savings	5
4	Business Model	5
4.1	Packaging	5
4.2	Revenue Drivers	6
5	Data Sources	6
6	System Architecture	7
7	Storage and Processing Design	8
7.1	Storage	8
7.2	Risk Processing	9
8	Product Delivery	10
9	Implementation and Repository Structure	10

10 Go-to-Market Difficulties	11
11 Scalability and Cost	12
12 Conclusion	13
Appendix A Prototype Commands	13

1 Introduction

This project proposes **Taiwan Semiconductor and Tech Stock Risk Dashboard**, a data product for retail investors who track Taiwan technology stocks but do not have time to monitor price movement, valuation, revenue, and financial statement signals every day. The product turns public market and fundamental data into an explainable risk score and a web dashboard. Its purpose is not to predict prices or recommend trades; it is a monitoring and research-efficiency tool that helps users decide which holdings deserve attention.

The first version focuses on 20 semiconductor and technology stocks, including TSMC, MediaTek, UMC, ASE Technology, Novatek, Realtek, Quanta, Delta Electronics, Hon Hai, and related hardware names. This universe is intentionally narrow: these stocks are familiar to Taiwan retail investors, have relatively available data, and are exposed to fast-moving market cycles where risk monitoring is valuable.

The implemented prototype contains a Python ingestion layer, a SQLite database, a rule-based risk scoring pipeline, a FastAPI backend, and a React/Vite frontend. The dashboard supports stock risk ranking, single-stock detail pages, factor-level risk comparison, and a portfolio watchlist view. Data can be pulled from FinMind when an API token is available, and the repository also includes deterministic demo data so the system can be reproduced for grading even without live API access.

2 Target Customer

The target customer is a **Taiwan retail investor who holds or follows 5 to 20 semiconductor and technology stocks**. This user is not a professional fund manager and usually does not have access to a Bloomberg Terminal, sell-side research workflow, or a dedicated analyst team. They may use brokerage apps, finance news sites, TradingView, Goodinfo, CMoney, or spreadsheet watchlists, but their daily process is fragmented.

A concrete persona is:

A 25–45 year-old Taiwan retail investor with a full-time job, a meaningful equity portfolio, and holdings tilted toward semiconductor, IC design, hardware, and electronics manufacturing stocks. They review holdings after work and want to know which stocks became riskier today and why.

This segment is the right initial wedge for three reasons. First, the technology-stock universe is narrow enough to build a focused product instead of a generic stock platform. Second, the customer pain point is frequent and repeated: market data changes daily, while revenue and financial statements change periodically. Third, the segment already pays attention to information products such as charting tools, stock screeners, and financial data sites, which makes willingness to pay more plausible than for a purely casual investor.

The status quo is manual and scattered. A user may check brokerage app prices, search news headlines, visit separate financial statement sites, and maintain a spreadsheet. This creates three practical problems:

- **Information overload:** the user sees many prices and headlines, but not a ranked list of risk changes.
- **Missed weak signals:** abnormal volume, drawdown, valuation pressure, and revenue deterioration may appear in different places.
- **No portfolio context:** most free tools focus on individual stocks rather than the user's watchlist concentration.

The proposed dashboard improves the status quo by aggregating the relevant signals into one risk-monitoring workflow. It does not replace deeper research; it prioritizes where deeper research should happen.

3 Evidence of Demand and Willingness to Pay

The demand claim is that retail investors will pay for tools that reduce the time cost of monitoring risky positions. I use three evidence paths: competitor pricing, public investor behavior research, and a reproducible local validation process that can be extended with surveys or forum scraping.

3.1 Competitor and Analogous Product Benchmarks

The strongest willingness-to-pay evidence is that individual investors already pay for financial data and charting products. TradingView, a global charting and market-data platform, publicly lists paid plans in addition to a free tier [1]. This shows that non-institutional investors are accustomed to subscription pricing for better monitoring, charting, and alerts. In Taiwan, products such as CMoney, Goodinfo, and other local stock tools occupy adjacent categories: screeners, financial data lookup, watchlists, and investor education. The proposed product is narrower than these platforms but more focused on one job: daily risk monitoring for Taiwan technology stocks.

Based on these benchmarks, the first commercial hypothesis is a freemium model:

- **Free:** track up to 5 stocks with basic risk score and daily ranking.
- **Pro:** NT\$199–399 per month for 20+ tracked stocks, portfolio watchlist, factor breakdown, and alert history.
- **Creator/community:** NT\$1,000–3,000 per month for a shareable watchlist dashboard used by investment communities or finance content creators.

This price range is intentionally below professional platforms. The product competes on focus and ease of interpretation, not on breadth of market data.

3.2 Behavioral Evidence

Retail investors actively use social and digital sources for investment decisions. Prior research on retail investor discussion platforms shows that investor conversations can contain useful signals and that retail investors seek digestible decision support from public information streams [2]. This supports the broader demand assumption: the target customer does not only want raw data; they want processed, interpretable signals.

For this project, the validation process is designed to be reproducible:

1. Identify the target segment: Taiwan retail investors following semiconductor or technology stocks.
2. Benchmark competing information products and their pricing.
3. Collect user pain points through a short questionnaire or investment-community posts.
4. Map repeated pain points to product features: risk ranking, factor breakdown, and portfolio concentration.
5. Estimate willingness to pay by comparing time saved and existing subscription prices.

3.3 Estimated Time Savings

A manual workflow for 10 holdings can easily require 20–40 minutes per day: checking price movement, reading headlines, checking volume spikes, and reviewing whether financial indicators changed. If the dashboard reduces this to a 5-minute risk scan, it saves roughly 15–35 minutes per active market day. For a user who values after-work research time at NT\$200 per hour, this implies NT\$1,000 or more in monthly time value during active monitoring periods. A subscription at NT\$199–399 per month is therefore plausible if the product is trusted and reliable.

The evidence is not yet enough to prove product-market fit. It is enough for a defensible course-project claim: the problem is real, analogous products are monetized, and the prototype targets a repeated workflow with measurable time savings.

4 Business Model

The monetized unit is not raw stock data. The customer pays for **risk interpretation, workflow compression, and portfolio monitoring**. The product bundles public data into a ranked daily view that answers: which stocks in my watchlist became riskier, which factor caused the change, and where should I research next?

4.1 Packaging

The proposed business model is subscription-based freemium:

- **Free tier:** 5 tracked stocks, latest risk score, and basic overview page.
- **Pro tier:** NT\$199–399 per month, 20+ tracked stocks, portfolio watchlist, factor-level ranking, and historical score trends.
- **Community tier:** NT\$1,000–3,000 per month, shareable dashboards for finance creators, investment clubs, or paid communities.

The pricing is designed for retail affordability. A high price would force the product to compete with professional research terminals; a low-to-mid subscription lets the product compete with existing retail finance apps and newsletters.

4.2 Revenue Drivers

Revenue depends on three drivers:

1. **Tracked-stock depth:** users are more likely to pay when the product covers enough names in their watchlist.
2. **Trust in explanations:** each risk score must show its component drivers, not just a black-box number.
3. **Retention through alerts:** daily or weekly risk changes create repeated usage.

The initial go-to-market channel would be finance communities, university investment clubs, Reddit-like forums, and finance content creators. The strongest early wedge is a narrow public demo: “20 Taiwan technology stocks ranked by explainable risk.” If users ask to add more stocks, alerts, or portfolio import, that becomes demand evidence for the next version.

5 Data Sources

The prototype is designed around public Taiwan equity data, with FinMind as the primary ingestion interface. FinMind provides Taiwan market datasets such as stock price, valuation, monthly revenue, and financial statements through documented APIs [3, 4]. The code also includes deterministic demo data so the project can be reproduced without an API token.

Source	Dataset	Use	Refresh
FinMind	TaiwanStockPrice	OHLC, volume, trading value	Trading day
FinMind	TaiwanStockPER	PER, PBR, dividend yield	Trading day or available
FinMind	TaiwanStockMonthRevenue	Monthly revenue, MoM, YoY	Monthly
FinMind	TaiwanStockFinancialStatements	EPS, profit, income metrics	Quarterly
Local generator	Deterministic demo seed	Reproducible demo and tests	On demand

Table 1: Data sources used by the prototype.

The source strategy deliberately separates data frequency from system refresh frequency. Market data can update every trading day after market close. Monthly revenue and financial statements do not change daily, but the pipeline can still check for new values daily and reuse the latest available values in the risk score.

The main data ethics issue is that the product should not overstate data quality or legal certainty. Public market data must be used according to the data provider’s terms. The first prototype avoids scraping social media or news sites because those sources introduce copyright, terms-of-service, and privacy risks.

6 System Architecture

The system is a batch-updated data product. It does not compute expensive analytics during each user request. Instead, ingestion and scoring happen in scheduled jobs, and the API serves precomputed results to the dashboard.

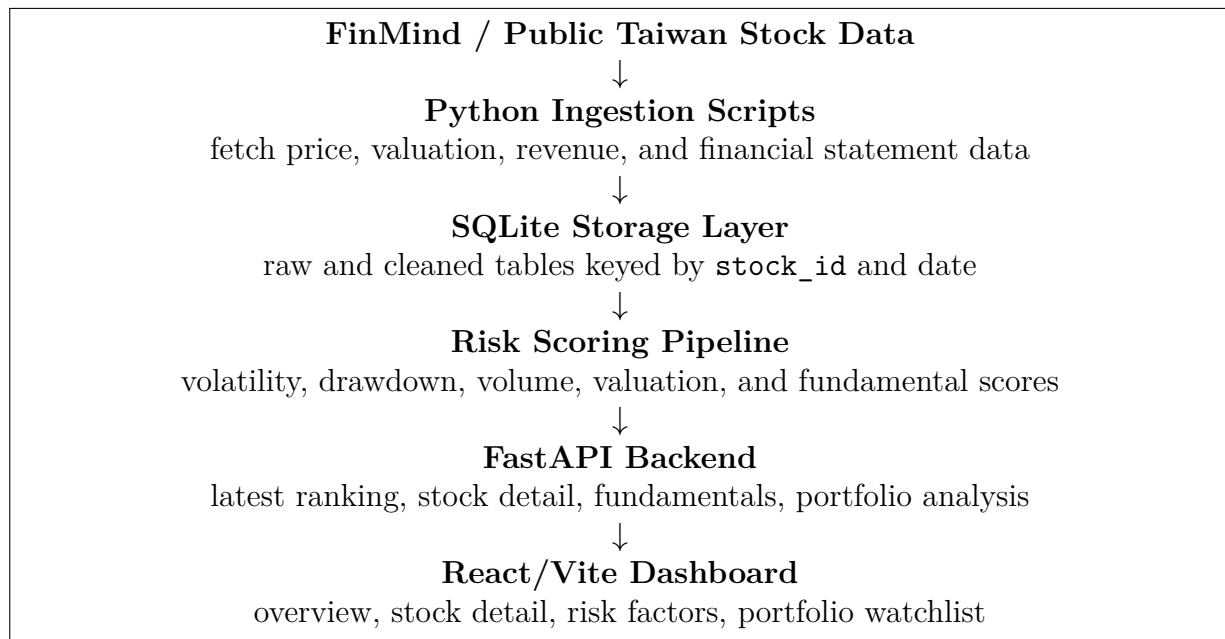


Figure 1: End-to-end data monetization system architecture.

The architecture has four main layers:

1. **Ingestion:** Python scripts call FinMind datasets and upsert rows into SQLite. If live data is unavailable, the demo seed generator creates deterministic sample data for reproducibility.
2. **Storage:** SQLite stores the tracked stock universe, market data, valuation metrics, monthly revenue, financial statement metrics, risk scores, and update logs.
3. **Processing:** the risk pipeline computes component scores and a total score for each tracked stock after data updates.
4. **Delivery:** FastAPI exposes JSON endpoints. React/Vite renders the user-facing dashboard.

This design keeps the prototype simple while preserving the shape of a real data product: a scheduled batch pipeline, persistent storage, precomputed analytical features, and a low-latency serving layer.

7 Storage and Processing Design

7.1 Storage

The prototype uses SQLite because the first version tracks only 20 stocks and needs to be easy to reproduce in a course environment. The schema contains seven primary tables:

- **stocks:** fixed stock universe, stock names, industry labels, and categories.

- `daily_prices`: OHLC, volume, and trading value by stock and date.
- `valuation_metrics`: PER, PBR, and dividend yield.
- `monthly_revenue`: revenue, month-over-month growth, and year-over-year growth.
- `financial_metrics`: EPS, gross profit, operating income, and net income.
- `risk_scores`: total and component risk scores.
- `update_log`: update attempts and fallback status.

The main join key is `stock_id`. Date is part of the primary key for time-series tables. This schema is intentionally normalized enough to support traceability, but not so complex that local setup becomes fragile.

7.2 Risk Processing

The dashboard uses an explainable rule-based score:

$$\text{Total Risk Score} = 0.25V + 0.25D + 0.20Q + 0.15P + 0.15F$$

where:

- V is volatility risk: the percentile rank of latest 20-day annualized volatility.
- D is drawdown risk: the current drawdown from the 60-day high, scaled so a 35% draw-down maps to 100.
- Q is abnormal-volume risk: latest volume relative to the 20-day average.
- P is valuation risk: average percentile rank of PER and PBR versus available history.
- F is fundamental risk: deterioration in monthly revenue YoY and recent EPS, gross profit, or net income.

Each component is clipped to a 0–100 range. The total score maps to three levels: Low for 0–39, Medium for 40–69, and High for 70–100. Missing data does not crash the pipeline; unavailable components fall back to a neutral score of 35. This is important because low-frequency fundamental data may be missing even when daily market data is available.

The design favors interpretability over model sophistication. A machine learning model might appear more advanced, but it would be harder to validate, easier to overfit, and more likely to be mistaken for investment advice.

8 Product Delivery

The customer consumes the product through a web dashboard. The prototype frontend is implemented with React and Vite, while the backend is implemented with FastAPI. The dashboard has four main views:

1. **Overview:** ranks the 20 tracked stocks by latest total risk score, shows the number of high-risk and medium-risk stocks, and displays a bar chart of the highest-risk names.
2. **Stock Detail:** shows price, volume, risk score trend, risk factor breakdown, latest valuation metrics, monthly revenue, and financial statement metrics for one stock.
3. **Risk Factors:** ranks stocks separately by volatility, drawdown, abnormal volume, valuation, and fundamental risk.
4. **Portfolio Watchlist:** lets the user enter holdings and weights, then computes a portfolio-level risk score and industry exposure.

The key workflow is:

Open the overview page, identify stocks with elevated risk, click into a stock detail page, inspect which factor drove the score, and decide whether deeper research is needed.

The API supports this workflow through endpoints such as `/api/risk/latest`, `/api/stocks/{stock_id}/summary`, `/api/risk/history/{stock_id}`, and `/api/portfolio/analyze`. The user-facing output is not a trading signal. It is a ranked risk explanation.

For demonstration, the repository includes a “Seed Demo Data” flow so graders can run the dashboard without a FinMind token. With a token, the same pipeline can update from live FinMind data.

9 Implementation and Repository Structure

The GitHub repository contains both the report and the working prototype. The implementation is organized as follows:

Path	Purpose
<code>backend/app</code>	FastAPI app, SQLite schema, ingestion modules, stock universe, and risk processing.
<code>frontend/src</code>	React dashboard pages and shared UI components.
<code>scripts</code>	Local commands for initializing the database, seeding demo data, updating data, and computing scores.
<code>tests</code>	Backend tests for risk score thresholds and API behavior.
<code>contents</code>	LaTeX report sections.
<code>.codex/skills</code>	Reusable semantic-layer notes for this project.

Table 2: Repository structure.

The local run path is:

1. Install backend dependencies from `backend/requirements.txt`.
2. Run `python scripts/seed_demo_data.py`.
3. Run `python scripts/compute_risk_scores.py`.
4. Start the API with `uvicorn backend.app.main:app --reload`.
5. Install frontend dependencies and run `npm run dev` in `frontend`.

The prototype was validated with backend tests and a frontend production build. The backend test suite checks that risk scores stay within expected ranges, thresholds map correctly to risk levels, the latest-risk API returns the 20-stock universe after seeding, and unknown stocks return a clear error. The frontend builds successfully with Vite.

10 Go-to-Market Difficulties

The main go-to-market difficulty is trust. Investors are skeptical of black-box tools, especially in financial contexts. The product should therefore avoid language such as “prediction,” “alpha,” or “buy signal.” It should show each score’s component breakdown and cite the latest data update time. Trust is earned through transparency, not through exaggerated claims.

The second difficulty is data licensing and reliability. FinMind is a useful development source, but a commercial version would need to review provider terms, data redistribution rights, uptime, rate limits, and whether official TWSE or MOPS data should be integrated directly. If news or social sentiment is added later, terms-of-service and copyright risk become much more important.

The third difficulty is competition. Broad platforms such as TradingView already provide charting and alerts [1]. Local Taiwan finance tools already provide financial statement lookup and stock screening. The proposed product must avoid competing on breadth. Its moat would come from a focused workflow: explainable risk monitoring for Taiwan technology-stock watchlists.

The fourth difficulty is regulation and liability. Even though the prototype is not investment advice, users may interpret risk rankings as trading recommendations. The product must include disclaimers, avoid personalized trade advice, and frame outputs as research prioritization.

Finally, unit economics depend on scale. A small prototype using SQLite and public APIs is cheap. A commercial product would incur costs for reliable data access, hosting, monitoring, alert delivery, customer support, and possibly legal review. The first profitable niche is likely a small paid community or creator plan rather than mass-market consumer acquisition.

11 Scalability and Cost

At the initial scale, SQLite is sufficient. With 20 stocks, daily price history, valuation rows, monthly revenue, and quarterly financial statements, the data volume is small. The main cost is developer time, not infrastructure.

Scale	Expected Behavior	Likely Change Needed
Initial	20 stocks, local SQLite, manual or scripted update	Current architecture is sufficient.
10x	200 stocks, daily batch, more frontend filtering	Add background scheduler, indexes, and better API pagination.
100x	2,000+ symbols or multiple markets	Move to PostgreSQL or warehouse storage; add job queue and observability.

Table 3: Scalability plan.

The first bottleneck is not computation. Risk scoring is simple pandas batch processing. The first real bottlenecks are data-provider limits, deployment persistence, and operational reliability. If the backend runs on Render with SQLite, persistent storage may be fragile. A production deployment should use PostgreSQL or another managed database.

The rough monthly infrastructure cost for a prototype can be close to zero using free tiers. A more reliable deployment might cost:

- US\$5–15 for backend hosting.
- US\$0–20 for database hosting at small scale.
- US\$0 for frontend hosting on a free Vercel tier.
- Additional cost for paid data access if public sources are insufficient.

The product becomes economically attractive only if the subscription revenue exceeds data and support costs. For example, 100 Pro users at NT\$299 per month would generate about NT\$29,900 per month before payment fees and operating costs. This is enough for a small side-project product, but not enough for a venture-scale company.

12 Conclusion

This project demonstrates how public financial data can be refined into a monetizable data product. The target customer is a Taiwan retail investor who follows semiconductor and technology stocks and needs a faster way to monitor risk. The product creates value by compressing scattered market and fundamental signals into an explainable dashboard.

The technical system implements the full data product loop: ingestion, storage, processing, API delivery, and frontend consumption. The risk score is intentionally rule-based and transparent, combining volatility, drawdown, abnormal volume, valuation, and fundamental deterioration. The dashboard then exposes those scores through overview, stock detail, factor ranking, and portfolio watchlist views.

The strongest business case is not that the product can beat the market. The stronger claim is that it can save research time, reduce missed risk signals, and help users prioritize deeper analysis. The main remaining risks are trust, data licensing, competition, and the need to validate willingness to pay with real users.

The next step would be to run a small user validation study with Taiwan retail investors, deploy the prototype publicly, and measure which features create repeat usage: daily ranking, alerts, portfolio risk, or stock-detail explanations.

Appendix A Prototype Commands

The prototype can be reproduced with the following local commands:

```
python scripts\seed_demo_data.py
python scripts\compute_risk_scores.py
uvicorn backend.app.main:app --reload
```

```
cd frontend
npm install
npm run dev
```

The backend tests can be run with:

```
python -m pytest tests
```

References

- [1] TradingView, “Tradingview pricing plans.” <https://www.tradingview.com/pricing/>, 2026. Accessed 2026-06-19.
- [2] R. Sawhney, A. Wadhwa, S. Agarwal, and R. R. Shah, “Stockbabble: A conversational financial agent to support stock market investors,” *arXiv preprint arXiv:2106.08298*, 2021.

- [3] FinMind, “Taiwan market technical data documentation.” <https://finmind.github.io/tutor/TaiwanMarket/Technical/>, 2026. Accessed 2026-06-19.
- [4] FinMind, “Taiwan market fundamental data documentation.” <https://finmind.github.io/tutor/TaiwanMarket/Fundamental/>, 2026. Accessed 2026-06-19.